

Especificación Técnica de Arquitectura: Agente de IA Autónomo Omnicanal (Nivel Enterprise)

Este documento es la **guía maestra y blueprint de ingeniería** para transformar el repositorio `ManyChat` (actualmente en `backend/server.js`) en un **Agente de IA Autónomo Omnicanal de Grado Empresarial** para la plataforma de ingeniería y consultoría `clipop.com.mx`.

Cualquier agente desarrollador o ingeniero de software debe seguir este documento al pie de la letra, respetando los contratos de interfaces, esquemas de datos y medidas de seguridad aquí descritas.

1. Visión General del Sistema y Objetivos

1.1. Objetivos Principales

- Desacoplar el monolito:** Reducir `server.js` (actualmente 1,600+ líneas) a un entrypoint limpio (< 90 líneas), organizando el código bajo una arquitectura modular limpia (*Clean Architecture*).
- Latencia y Resiliencia en Ingesta (< 500 ms):** Responder a Meta con `HTTP 200 EVENT_RECEIVED` de inmediato. Todo el razonamiento del agente debe ejecutarse en segundo plano mediante un pipeline asíncrono.
- Multimodalidad Total (Voz y Visión):**
 - Transcripción ultrarrápida (< 500 ms) de notas de voz de WhatsApp e Instagram utilizando **Groq Whisper-large-v3**.
 - Respuestas en notas de voz nativas de WhatsApp (formato OGG Opus) mediante **OpenAI TTS / ElevenLabs** si el usuario interactuó con audio.
 - Análisis de imágenes técnicas (planos de CFE, diagramas unifilares, cotizaciones, recibos) con la visión de **Gemini 1.5 Flash**.
- Razonamiento con Herramientas (*Tool Calling / Function Calling*):**
 - El modelo no es un simple generador de texto; ejecuta funciones reales en el backend para cotizar proyectos, agendar llamadas en Google Calendar, buscar cursos en la base de datos y transferir chats a agentes humanos.
- Memoria Semántica RAG con `pgvector` :**

- PostgreSQL con extensión vectorial para indexar normas de CFE, especificaciones de precios unitarios OPUS/Neodata y el catálogo de cursos.
-

2. Diagrama de Secuencia de la Arquitectura

```
sequenceDiagram
    autonumber
    actor U as Usuario (WhatsApp / IG / FB)
    participant GW as Gateway / Webhook Listener
    participant DB as PostgreSQL (Prisma)
    participant MM as Capa Multimodal (Groq Whisper / Gemini Vision)
    participant AG as Agente Core (Gemini 1.5 Flash)
    participant RAG as Vector Store (pgvector)
    participant TC as Herramientas (Tools Backend)
    participant SY as Sintetizador (Texto / TTS Opus)
    participant DP as Despachador (Meta API / Baileys)

    U->>GW: Envía mensaje (Texto, Audio o Imagen)
    Note over GW: 1. Valida firma & deduce duplicados (MID cache)
    GW-->>U: HTTP 200 OK Inmediato (Cumple SLA Meta)
    GW->>DB: Registra / Actualiza Lead & Guarda Mensaje 'user'

    rect rgb(240, 248, 255)
        Note over GW,MM: Preprocesamiento de Contenido
        alt Si el mensaje es Nota de Voz
            GW->>MM: Envía buffer de audio
            MM->>MM: Groq Whisper transcribe audio a texto
        else Si el mensaje es Imagen / Plano
            GW->>MM: Descarga imagen y extrae OCR/descripción
        end
    end

    Note over GW,AG: Validación de Intervención Humana
    GW->>DB: ¿Lead.bot_paused == true?
    alt Bot Pausado por Agente Humano
        GW->>GW: Aborta respuesta automática (Humano atiende)
    else Bot Activo
        GW->>RAG: Búsqueda de contexto semántico relevante (Top 3 Chunks)
        RAG-->>AG: Inyecta fragmentos de normas y cursos
        GW->>AG: Invoca Agente (Historial + Prompt + Tools Declaradas)

        loop Ciclo de Razonamiento (Tool Calling)
            AG->>AG: Decide si necesita invocar una herramienta
            alt Requiere ejecutar función
                AG->>TC: Invoca Tool (ej. cotizarProyecto, agendarCita)
                TC->>DB: Lee/Escribe datos en base de datos
                TC-->>AG: Retorna resultado JSON de la función
            end
        end
    end

    AG-->>SY: Genera respuesta final

    alt Si usuario envió audio originalmente
        SY->>MM: Sintetiza texto a voz (TTS Opus PTT)
        MM-->>DP: Buffer de audio OGG
        DP->>U: Envía Nota de Voz nativa
```

```
else Si usuario envió texto
  SY→DP: Formatea Markdown (limpio y emojis)
  DP→U: Envía Mensaje de Texto
end

DP→DB: Guarda respuesta 'ai' en Conversation
end
```

3. Estructura de Directorios del Backend (Modular y Desacoplada)

Toda la lógica monolítica de `server.js` debe reorganizarse estrictamente en esta jerarquía:

```

backend/
├─ prisma/
│  └─ schema.prisma          # Esquema de datos con pgvector
│  └─ migrations/            # Historial de migraciones
├─ src/
│  └─ config/
│     └─ env.js              # Validación estricta de variables de entorno
│     └─ database.js         # Pool de conexión de PostgreSQL y Prisma Client
│     └─ constants.js        # Constantes globales del sistema
│  └─ middlewares/
│     └─ errorHandler.js     # Middleware global de errores Express
│     └─ metaSignature.js    # Validación criptográfica SHA256 para webhooks de Meta
│  └─ routes/
│     └─ metaWebhook.routes.js # GET (verificación) y POST (eventos de Meta)
│     └─ whatsapp.routes.js   # Estado, QR y Pairing code de Baileys
│     └─ leads.routes.js      # API de Leads y Live Chat para el frontend
│     └─ products.routes.js   # CRUD de cursos y catálogo
│     └─ settings.routes.js   # Configuración dinámica del bot y credenciales
│  └─ controllers/
│     └─ metaWebhook.controller.js
│     └─ whatsapp.controller.js
│     └─ leads.controller.js
│  └─ services/
│     └─ metaGraphService.js  # Cliente HTTP para envíos a WhatsApp Cloud y Graph API
│     └─ baileysService.js    # Conexión por socket y eventos de Baileys
│  └─ agent/                  # ← NÚCLEO COGNITIVO DEL AGENTE
│     └─ agentCore.js         # Orquestador del ciclo de vida del agente
│     └─ systemPrompts.js     # Prompts base, personalidad y directrices de CLIPOP
│     └─ tools/               # Herramientas ejecutables (Function Calling)
│        └─ toolDefinitions.js # Declaraciones JSON Schema para Gemini
│        └─ quotationTool.js   # Lógica de cotización de proyectos de alta/media tensión
│        └─ calendarTool.js    # Lógica de agendamiento de citas técnicas
│        └─ catalogTool.js     # Consulta de cursos de OPUS/Neodata/CFE
│        └─ handoverTool.js    # Pausa de bot y escalamiento a operador humano
│     └─ multimodal/
│        └─ speechToText.js    # Integración Groq Whisper / Gemini Audio
│        └─ textToSpeech.js    # Síntesis TTS en formato Opus PTT
│        └─ visionService.js   # Procesamiento de planos y fotos con Gemini Vision
│     └─ rag/
│        └─ vectorStore.js     # Consultas de similitud coseno con pgvector
│        └─ embeddings.js      # Generación de vectores con text-embedding-004
│  └─ app.js                  # Configuración y middlewares de Express
├─ ecosystem.config.js        # Configuración de PM2 en Google Cloud
├─ package.json               # Dependencias
└─ server.js                  # Entrypoint simple (< 90 líneas)

```

4. Esquema de Base de Datos Mejorado (`schema.prisma`)

Se debe habilitar la extensión `vector` en PostgreSQL y expandir los modelos para soportar la memoria semántica, las herramientas y la auditoría de costos/tokens:

```

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  extensions = [vector]
}

generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions"]
}

// 1. Catálogo de Cursos y Servicios
model Product {
  id          Int      @id @default(autoincrement())
  name        String
  description  String?
  long_description String? @db.Text
  competencias String[] // Temas clave del curso
  price       Float
  image_url   String?
  link        String?
  is_active   Boolean @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

// 2. Leads (Prospectos) con atributos de IA y canal
model Lead {
  id          Int      @id @default(autoincrement())
  platform    String    // "whatsapp", "instagram", "messenger", "web"
  phone_or_id String    @unique
  name        String?
  email       String?
  status      String    @default("NUEVO") // "NUEVO", "EN_CONTACTO", "COTIZADO", "CERRADO"
  bot_paused  Boolean    @default(false)  // Toma de control humana
  prefers_audio Boolean @default(false)    // Si el usuario prefiere notas de voz
  sentiment   String?    // "interesado", "molesto", "neutral", "urgente"
  custom_data Json?      // Datos recolectados por tools (ej: presupuesto, tipo de obra)
  conversations Conversation[]
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

// 3. Conversaciones con soporte multimodal y trazabilidad de tools
model Conversation {
  id          Int      @id @default(autoincrement())
  leadId      Int
  lead        Lead     @relation(fields: [leadId], references: [id], onDelete: Cascade)
  message     String    @db.Text
  sender      String    // "user", "ai", "human"
  media_type   String    @default("text") // "text", "audio", "image", "document"
  media_url    String?   // URL del archivo o nota de voz
  tool_invoked String?   // Nombre de la tool si la IA ejecutó una acción
  tokens_used  Int?      // Conteo de tokens para métricas de costo
}

```

```

    timestamp    DateTime @default(now())
}

// 4. Base de Conocimiento RAG Vectorial (pgvector)
model DocumentChunk {
    id            Int            @id @default(autoincrement())
    title         String         // Ej: "Norma CFE Contratación", "Temario OPUS 2025"
    category      String         // "normas_cfe", "cursos_opus", "politicas"
    content       String         @db.Text
    embedding     Unsupported("vector(768)")? // Vector de 768 dimensiones para Gemini text-embedding-004
    createdAt     DateTime       @default(now())
    updatedAt     DateTime       @updatedAt

    @@index([category])
}

// 5. Configuraciones Dinámicas del Sistema
model Setting {
    id            Int            @id @default(autoincrement())
    key           String         @unique
    value         String         @db.Text
    description   String?
    updatedAt     DateTime       @updatedAt
}

// 6. Reglas Rápidas de Palabras Clave (Short-circuit antes de IA)
model BotRule {
    id            Int            @id @default(autoincrement())
    keyword       String
    match_type    String         @default("contains") // "contains", "exact"
    response      String         @db.Text
    is_active     Boolean        @default(true)
    createdAt     DateTime       @default(now())
    updatedAt     DateTime       @updatedAt
}

```

5. Especificación de Componentes Clave

5.1. Capa de Ingesta y Webhook Oficial de Meta (`metaWebhook.controller.js`)

[!CRITICAL] Para evitar tormentas de reenvíos (*re-delivery storms*) por parte de los servidores de Meta, el webhook **DEBE responder con HTTP 200 inmediatamente** antes de invocar cualquier llamada a base de datos o LLM.

```
// Algoritmo del receptor del Webhook:
async function handleMetaIncoming(req, res) {
  // 1. Responder a Meta inmediatamente
  res.status(200).send('EVENT_RECEIVED');

  const { body } = req;
  // 2. Extraer plataforma, ID de mensaje (mid) y timestamp
  const { platform, mid, phoneOrId, senderName, messageText, media, timestamp } = parseMetaPayload(body);

  // 3. Filtrar duplicados usando cache TTL en memoria o Redis
  if (isDuplicateMessage(mid)) {
    return console.log(`[Deduplicator] Mensaje duplicado ignorado: ${mid}`);
  }

  // 4. Encolar el procesamiento para ejecución asíncrona
  agentWorkerQueue.push({ platform, phoneOrId, senderName, messageText, media, timestamp });
}
```

5.2. Módulo Multimodal: Transcripción de Audio y Síntesis de Voz

A. Transcripción de Notas de Voz (`speechToText.js`):

Cuando un usuario envíe una nota de voz por WhatsApp o Messenger:

1. Extraer el buffer del audio desde Baileys (`downloadMediaMessage`) o desde la URL firmada de Meta Graph API.
2. Enviar el stream a la API de **Groq Whisper** (`whisper-large-v3`):

```
const Groq = require('groq-sdk');
const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });

async function transcribeAudioBuffer(audioBuffer, filename = 'voice_note.ogg') {
  const file = await Groq.toFile(audioBuffer, filename, { type: 'audio/ogg' });
  const transcription = await groq.audio.transcriptions.create({
    file: file,
    model: 'whisper-large-v3',
    language: 'es',
    temperature: 0.0
  });
  return transcription.text;
}
```

B. Síntesis de Respuesta a Voz (`textToSpeech.js`):

Si el usuario interactuó con audio, el bot no debe contestar con un muro de texto; debe enviar una nota de voz nativa:

1. Se sintetiza el texto a audio con **OpenAI TTS** (`tts-1`) o **ElevenLabs**.

2. Se envía a Baileys indicando `ptt: true` (Push To Talk - nota de voz oficial de WhatsApp):

```
await sock.sendMessage(jid, {  
  audio: audioBuffer,  
  mimetype: 'audio/ogg; codecs=opus',  
  ptt: true // Esto hace que aparezca con el icono de micrófono verde en WhatsApp  
});
```

5.3. Declaración y Enrutamiento de Herramientas (*Function Calling*)

Las herramientas le dan al agente la capacidad de ejecutar código en el backend. Se declaran como esquemas JSON para el SDK de Google Generative AI:

```
// src/agent/tools/toolDefinitions.js
const toolsDeclaration = [
  {
    name: "cotizarProyecto",
    description: "Calcula y registra formalmente una solicitud de cotización técnica para proyectos de",
    parameters: {
      type: "OBJECT",
      properties: {
        tipoProyecto: {
          type: "STRING",
          description: "Tipo de obra (ej. 'Subestación eléctrica', 'Línea de media tensión', 'Obra ci",
        },
        voltajeNivel: {
          type: "STRING",
          description: "Nivel de tensión si se conoce (ej. '13.8 kV', '34.5 kV', '115 kV', 'No especi",
        },
        requiereCatalogoConceptos: {
          type: "BOOLEAN",
          description: "Si el cliente cuenta ya con catálogo de conceptos para revisión de precios un",
        },
        correoContacto: {
          type: "STRING",
          description: "Correo electrónico donde se enviará el expediente técnico de la cotización"
        }
      },
      required: ["tipoProyecto", "correoContacto"]
    }
  },
  {
    name: "agendarAsesoría",
    description: "Agenda una sesión técnica en vivo con un ingeniero de CLIPOP o bloquea un horario p",
    parameters: {
      type: "OBJECT",
      properties: {
        fechaDeseada: { type: "STRING", description: "Fecha en formato YYYY-MM-DD" },
        horaDeseada: { type: "STRING", description: "Hora aproximada (ej. '10:00 AM')" },
        tema: { type: "STRING", description: "Motivo de la reunión (ej. 'Duda sobre concurso CFE con"
      },
      required: ["fechaDeseada", "tema"]
    }
  },
  {
    name: "escalarAAgenteHumano",
    description: "Pausa el bot automático y transfiere la conversación a un ingeniero asesor humano c",
    parameters: {
      type: "OBJECT",
      properties: {
        motivo: { type: "STRING", description: "Razón por la cual se transfiere a un humano" },
        prioridad: { type: "STRING", enum: ["baja", "media", "alta", "urgente"] }
      },
      required: ["motivo"]
    }
  }
]
```

```
}  
];
```

Ejecución Dinámica de Herramientas en `agentCore.js` :

```
// Bucle de resolución de Tools:  
while (modelResponse.functionCalls && modelResponse.functionCalls.length > 0) {  
  for (const call of modelResponse.functionCalls) {  
    const { name, args } = call;  
    let toolResult = null;  
  
    if (name === "cotizarProyecto") {  
      toolResult = await quotationTool.execute(leadId, args);  
    } else if (name === "agendarAsesoría") {  
      toolResult = await calendarTool.execute(leadId, args);  
    } else if (name === "escalarAAgenteHumano") {  
      toolResult = await handoverTool.execute(leadId, args);  
    }  
  
    // Reinyectar el resultado de la función al chat para que el modelo genere la respuesta final al  
    chatSession.sendFunctionResult(name, toolResult);  
  }  
}
```

5.4. Memoria Semántica RAG con `pgvector`

Para consultas técnicas complejas (ej. normativas de CFE, diferencias de cálculo de FSR en OPUS 2025 vs 2024), el bot no alucinará:

```

-- Función SQL para buscar similitud coseno en PostgreSQL:
CREATE OR REPLACE FUNCTION match_documents (
    query_embedding vector(768),
    match_threshold float,
    match_count int
)
RETURNS TABLE (
    id int,
    title varchar,
    content text,
    similarity float
)
LANGUAGE sql STABLE
AS $$
    SELECT
        id,
        title,
        content,
        1 - (DocumentChunk.embedding <=> query_embedding) AS similarity
    FROM "DocumentChunk"
    WHERE 1 - (DocumentChunk.embedding <=> query_embedding) > match_threshold
    ORDER BY similarity DESC
    LIMIT match_count;
$$;

```

6. Variables de Entorno Requeridas (`.env.example`)

El nuevo agente requerirá la siguiente configuración en su archivo `.env` en producción (Google Cloud):

```

# Servidor y Entorno
NODE_ENV=production
PORT=3000

# Base de Datos PostgreSQL con extensión vector
DATABASE_URL="postgresql://usuario:password@localhost:5432/clipop_db?schema=public"

# Inteligencia Artificial (Google Gemini)
GEMINI_API_KEY="AIzaSy..."

# Multimodal Audio (Groq Whisper ultrarrápido)
GROQ_API_KEY="gsk_..."

# Multimodal Voz Saliente (Opcional: OpenAI TTS o ElevenLabs)
OPENAI_API_KEY="sk-proj-..."
ELEVENLABS_API_KEY=""

# Meta for Developers (WhatsApp Cloud API, Instagram, Messenger)
META_VERIFY_TOKEN="clipop_bot_verify_token_2026"
META_ACCESS_TOKEN="" # Guardado en DB o .env sin comillas
WHATSAPP_PHONE_NUMBER_ID=""

# Notificaciones a Agentes Humanos (Telegram Bot o Webhook de Slack)
ADMIN_ALERT_TELEGRAM_BOT_TOKEN=""
ADMIN_ALERT_CHAT_ID=""

```

7. Plan de Implementación Paso a Paso (Guía para el Desarrollador)

Paso 1: Limpieza de Credenciales Expuestas

1. Eliminar inmediatamente el fallback `DEFAULT_META_TOKEN` de la línea 1045 de `server.js`.
2. Crear archivo `.env.example` limpio en el repositorio.

Paso 2: Creación de la Estructura de Carpetas Modular

1. Crear el árbol de directorios `backend/src/{config,controllers,routes,services,middlewares,agent}`.
2. Mover la inicialización de base de datos a `src/config/database.js`.

Paso 3: Migración del Esquema Prisma

1. Habilitar `previewFeatures = ["postgresqlExtensions"]` y la extensión `vector` en `schema.prisma`.
2. Ejecutar `npm run prisma db push` o `npx prisma migrate dev --name init_agent_vector`.

Paso 4: Implementación del Módulo Multimodal

1. Instalar `groq-sdk` para transcripción de audio Whisper.
2. Extender `whatsappService.js` para detectar notas de voz y emitir buffers al manejador de mensajes.

Paso 5: Implementación del Orquestador del Agente (`agentCore.js`)

1. Implementar la clase `AgentOrchestrator` con soporte para:
 - Carga de historial de conversaciones desde Prisma.
 - Inyección del System Prompt oficial de CLIPOP.
 - Enlace con `toolsDeclaration`.
 - Ejecución de bucle de Function Calling.

Paso 6: Modularización de Rutas y Desacoplamiento de `server.js`

1. Extraer los endpoints de `server.js` hacia sus rutas correspondientes:
 - `/api/webhooks/meta` → `metaWebhook.routes.js`
 - `/api/whatsapp/*` → `whatsapp.routes.js`
 - `/api/leads/*` → `leads.routes.js`
 - `/api/products/*` → `products.routes.js`
 - `/api/settings` → `settings.routes.js`
2. Dejar `server.js` únicamente como punto de arranque con `app.listen()`.

8. Protocolo de Pruebas y Validación (Zero-Downtime)

Antes de reiniciar el servicio en Google Cloud con `pm2 restart clipop-backend` :

1. Prueba de Compilación y Sintaxis:

```
node --check backend/server.js
```

2. Prueba de Verificación de Webhook de Meta:

```
curl -X GET "http://localhost:3000/api/webhooks/meta?hub.mode=subscribe&hub.verify_token=clipop_b"
# Debe responder: CHALLENGE_TEST con status 200
```

3. Prueba de Ingesta Asíncrona (Simulación de Mensaje de WhatsApp):

- Enviar un mensaje de prueba al webhook local:

```
curl -X POST "http://localhost:3000/api/meta/test-message" \
-H "Content-Type: application/json" \
-d '{"phone": "526624745958", "message": "Hola, necesito cotizar un proyecto de media tensión en"
```

- Verificar en los logs de PM2 que el agente invoque la herramienta `cotizarProyecto` .

4. Prueba de Pausa de Bot (Intervención Humana):

- Activar `bot_paused = true` desde el panel web o por API.
- Enviar mensaje y verificar que el bot permanezca silenciado y el mensaje quede listo para ser respondido por el operador humano en `Chats.jsx` .